



CSIC 2026

GUSTAVO.LINAN@CSIC.ES

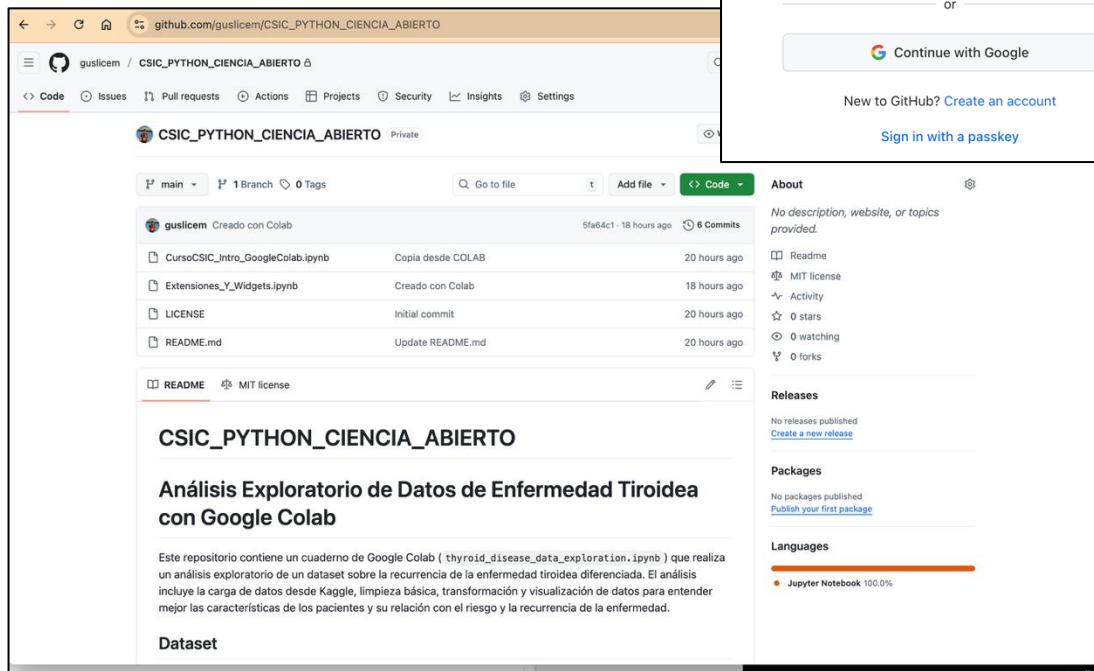
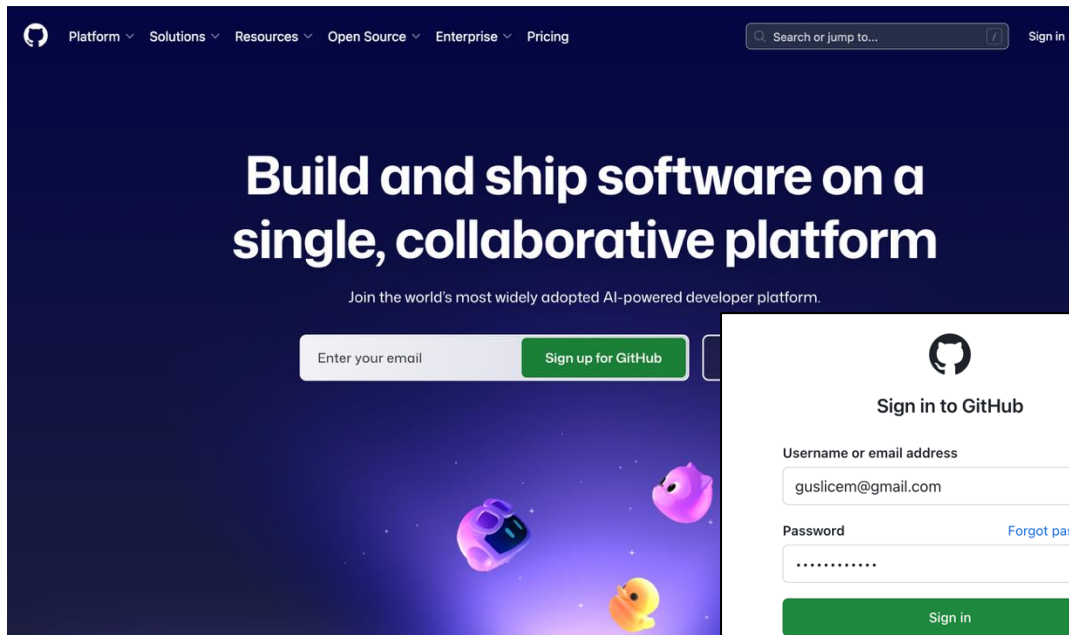
IMSE
-cnm



Instituto de
Microelectrónica
de Sevilla



Introducción a GitHub



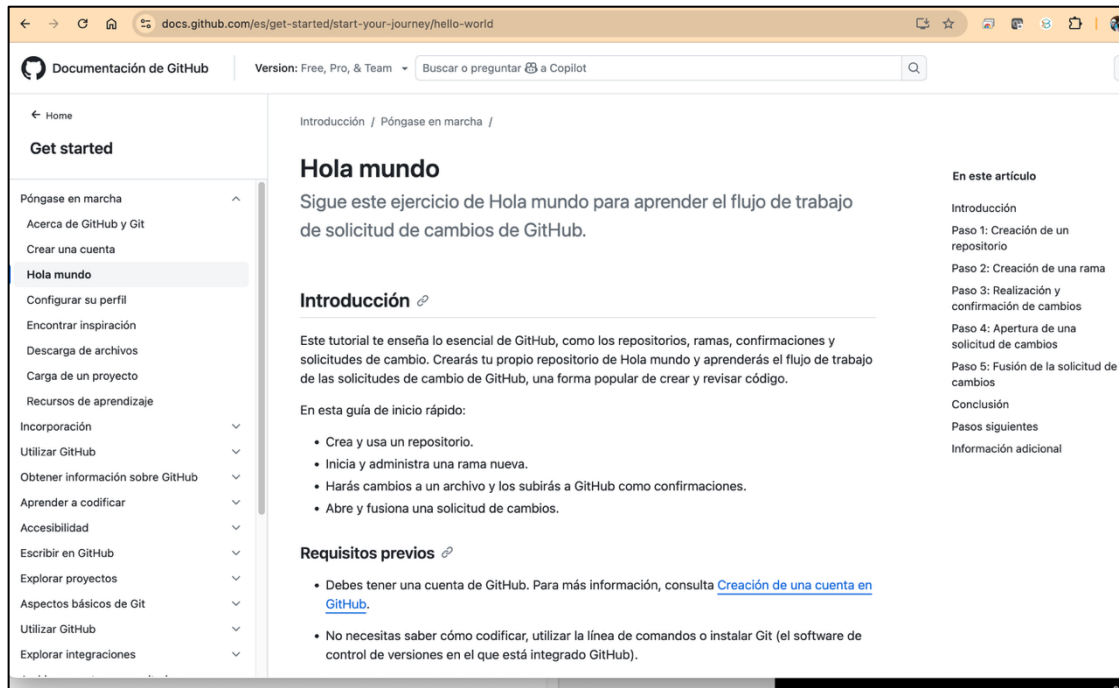
GitHub es una plataforma en la nube que se basa en **Git**, un sistema de control de versiones distribuido.

En el contexto de la ciencia abierta, GitHub resulta especialmente útil porque:

- Permite **compartir código** y datos de manera transparente.
- Facilita la **colaboración** entre investigadores distribuidos geográficamente.
- Mantiene un **historial detallado de cambios** en el código, asegurando la **trazabilidad** de resultados científicos.
- Hace posible **que otras personas propongan mejoras** (pull requests), reporten errores (issues) y reutilicen el trabajo de forma sencilla.



CONCEPTOS BÁSICOS



<https://docs.github.com/es/get-started/start-your-journey/hello-world>

- **Repositorio:** Es el “contenedor” del proyecto. Guarda todo el código, los datos, licencias, y ... sobre todo!!!! el historial de cambios.
- **Commit:** Una especie de “fotografía” del proyecto en un momento concreto. Cada vez que hacemos un commit, dejamos guardado un estado con un mensaje que describe el cambio.
- **Branch:** Una línea de desarrollo paralela. Permite experimentar o probar nuevas ideas sin modificar directamente la versión principal (normalmente la rama main o master).
- **Merge:** Proceso mediante el cual los cambios de una rama se integran en otra. Es la manera habitual de trabajar. Puede ser automático o requerir resolver conflictos si dos personas han modificado la misma parte del código.
- **Push:** Enviar los cambios locales (commits hechos en tu ordenador) al repositorio.
- **Pull:** "Traerte" los cambios remotos (los que tus colaboradores han subido a GitHub) a tu repositorio local.
- **Fork:** Copia completa de un repositorio ajeno en tu cuenta de GitHub. Permite trabajar de forma independiente en un proyecto sin alterar el repositorio original ni necesitar permisos directos de escritura.

Trabajar con GitHub

1) **Instalación de Git en local** y ejecución a través del terminal, es la forma más flexible y potente de usar Git, pero también la más compleja.

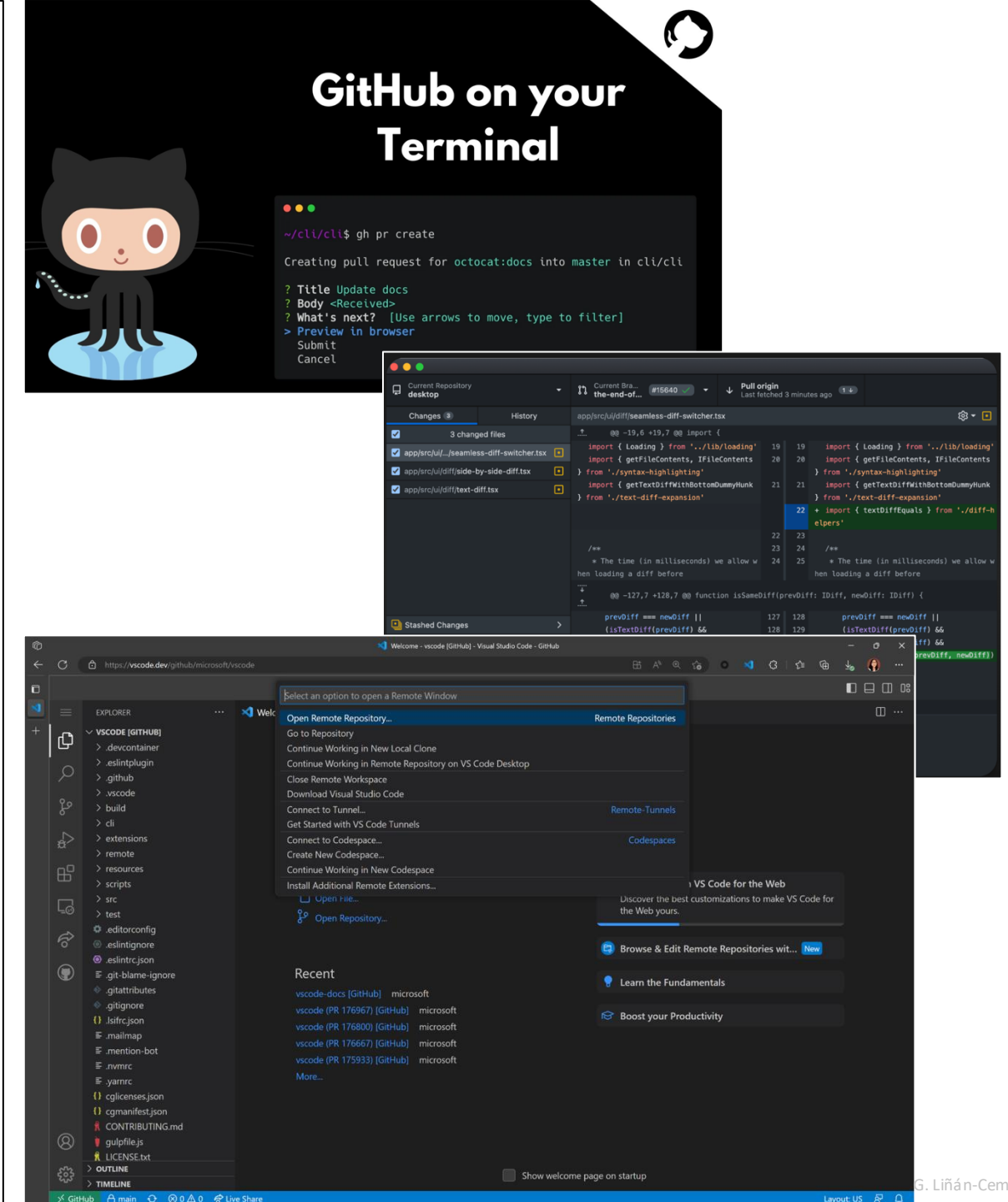
- MacOS: `brew install git`.
- Linux: `sudo apt install git`
- Windows: <https://git-scm.com/downloads>

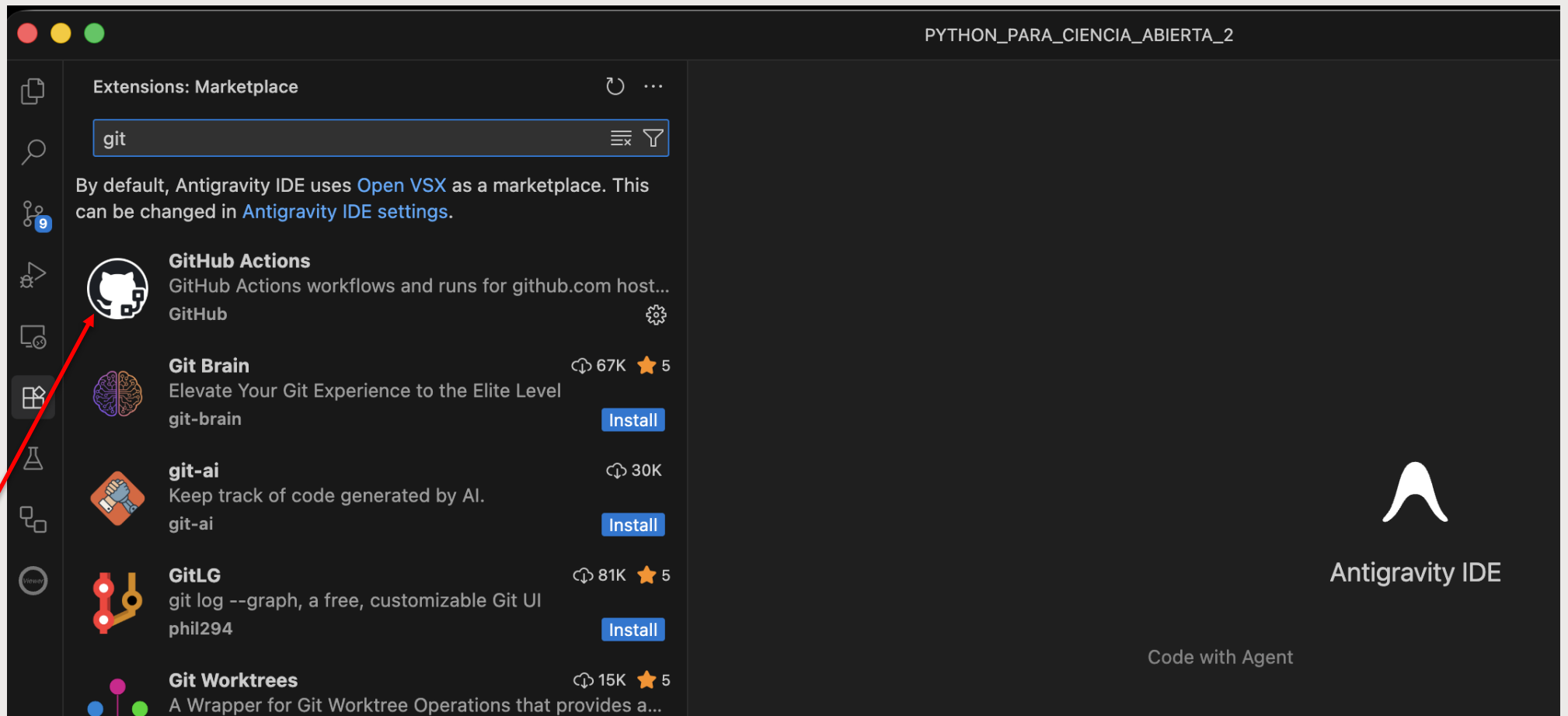
2) **Uso a través de GitHub Desktop:** Una aplicación gráfica oficial de GitHub, muy útil para quienes no desean trabajar con el terminal.

- Flujo básico:
- Instalar desde desktop.github.com.
- Iniciar sesión con tu cuenta de GitHub.
- Clonar o crear un repositorio nuevo.
- Realizar cambios en los archivos localmente.
- En GitHub Desktop: seleccionar los archivos modificados → escribir mensaje → Commit to main.
- Usar el botón Push origin para subir los cambios.

3) **Uso a través de la extensión para Antigravity**

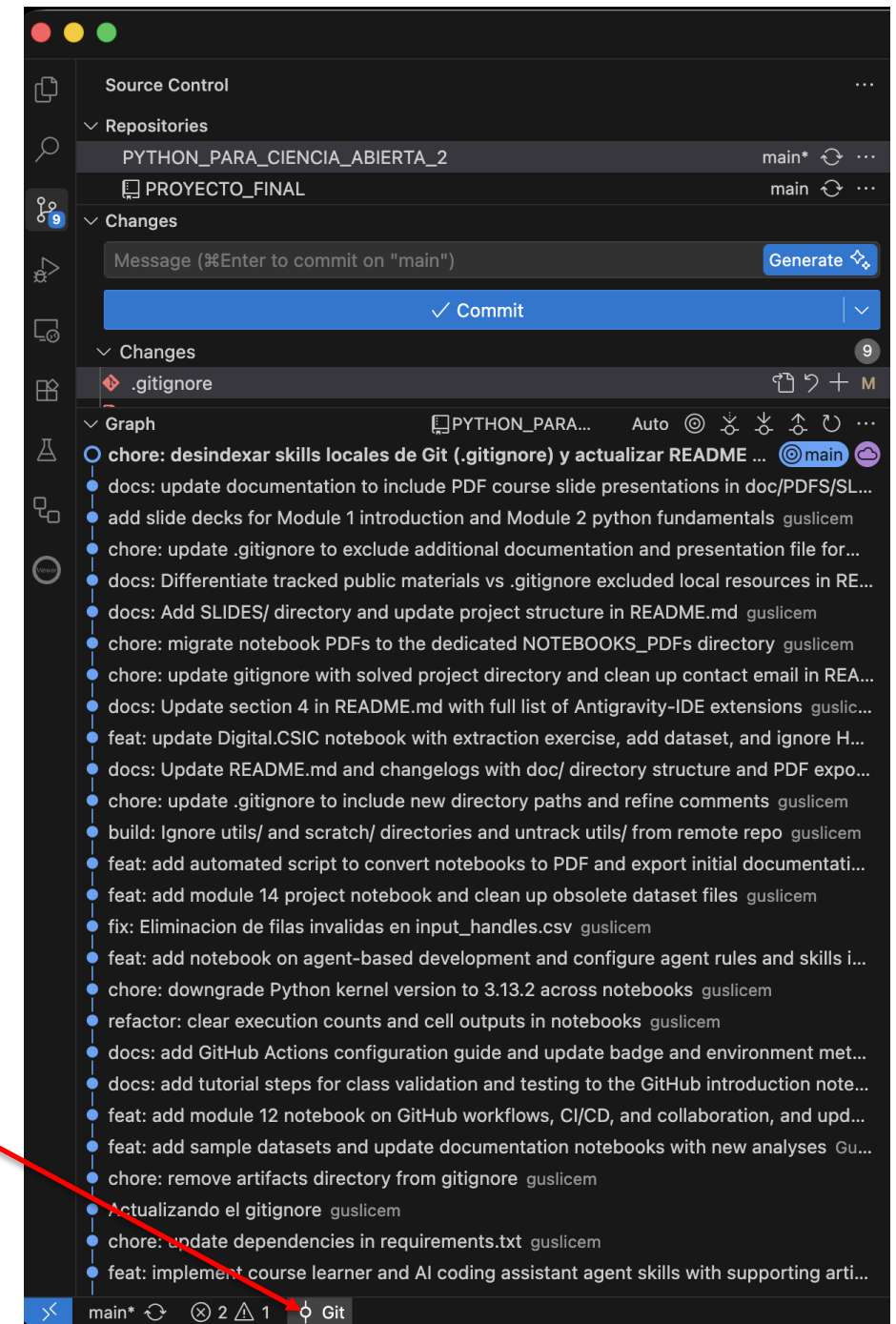
Antigravity-IDE integra Git de manera nativa y también tiene extensiones específicas para trabajar directamente con GitHub. Será nuestra opción en esta introducción: **Instala la extensión de Github Actions.**





Comprueba que has instalado la extension Github actions

Click aquí para observar el historial de cambios del repo



Algunos cambios mientras preparaba el curso

PYTHON_PARA_CIENCIA_ABIERTA_2 — Git History

Git History ×

Enter to search commit Repo: PYTHON_PARA_... Branch: All Author ↺ ↻ ⬇ ⬆ ⬇ 📄 🔍 ⚙️

Graph	Message	Date	Author	Hash
○	Uncommitted Changes (9)	now	*	*
●	main origin chore: desindexar skills locales de Git (.gitignore) y actualizar README y c...	2026-08-24 12:29	guslicem	f56e30a
●	docs: update documentation to include PDF course slide presentations in doc/PDFS/SLIDES/	2026-08-24 12:22	guslicem	ba947d1
●	add slide decks for Module 1 introduction and Module 2 python fundamentals	2026-08-24 12:19	guslicem	b78554b
●	chore: update .gitignore to exclude additional documentation and presentation file formats	2026-08-24 10:53	guslicem	06cfae5
●	docs: Differentiate tracked public materials vs .gitignore excluded local resources in README.md	2026-08-24 09:34	guslicem	98f6af0
●	docs: Add SLIDES/ directory and update project structure in README.md	2026-08-24 09:33	guslicem	67747fd
●	chore: migrate notebook PDFs to the dedicated NOTEBOOKS_PDFs directory	2026-08-24 09:32	guslicem	a6a7043
●	chore: update gitignore with solved project directory and clean up contact email in README	2026-08-24 09:09	guslicem	eb3f36a
●	docs: Update section 4 in README.md with full list of Antigravity-IDE extensions	2026-08-21 12:23	guslicem	4826424
●	feat: update Digital.CSIC notebook with extraction exercise, add dataset, and ignore HTML files	2026-08-21 12:04	guslicem	014a8ec
●	docs: Update README.md and changelogs with doc/ directory structure and PDF export utility	2026-08-21 11:19	guslicem	fce6c8e
●	chore: update .gitignore to include new directory paths and refine comments	2026-08-21 11:14	guslicem	e6e7861
●	build: Ignore utils/ and scratch/ directories and untrack utils/ from remote repo	2026-08-21 11:13	guslicem	6947861
●	feat: add automated script to convert notebooks to PDF and export initial documentation set	2026-08-21 11:09	guslicem	9d91b2c
●	feat: add module 14 project notebook and clean up obsolete dataset files	2026-08-21 11:02	guslicem	f8f0dc0
●	fix: Eliminacion de filas invalidas en input_handles.csv	2026-08-20 12:10	guslicem	c0d25b9
●	feat: add notebook on agent-based development and configure agent rules and skills in project root	2026-08-20 12:10	guslicem	b79c8e9
●	chore: downgrade Python kernel version to 3.13.2 across notebooks	2026-08-20 10:12	guslicem	5caae90

¿Y ahora qué?

En el cuaderno **12_INTRO_GITHUB.ipynb**:

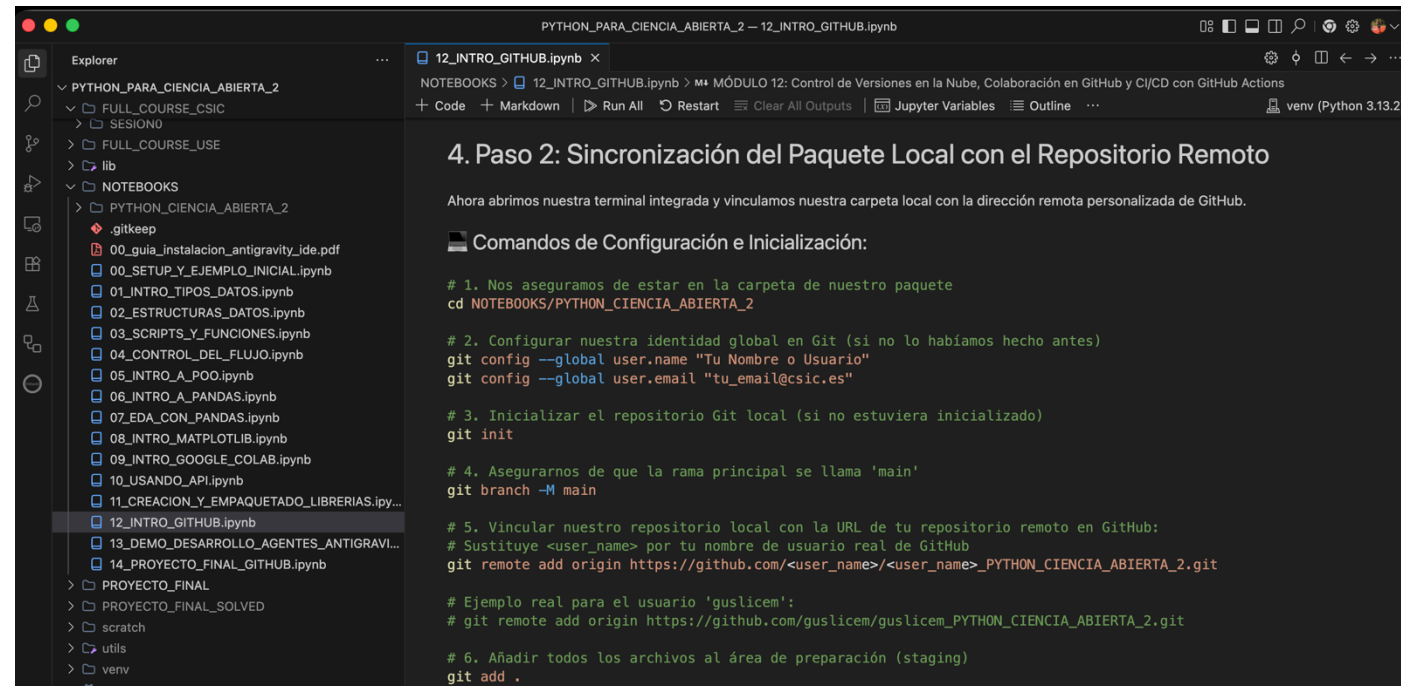
1.  **Tu Repositorio Remoto:** Crearás tu repositorio personal en la web de GitHub (<usuario>_PYTHON_CIENCIA_ABIERTA_2).
2.  **Vincular Local y Nube:** Conectaremos tu paquete local de Python con GitHub y subiremos tu primer commit oficial (git push).
3.  **Flujo de Ramas (*Feature Branch*):** Crearemos una rama de desarrollo (feature/edad-persona) para implementar nuevas reglas sin romper la rama principal.
4.  **Pull Request (PR):** Propondrás la integración de tus cambios a la rama principal simulando una revisión colaborativa.
5.  **Automatización con GitHub**
Actions: Activaremos la Integración Continua (CI/CD) para que GitHub pase los tests de pytest automáticamente en la nube tras cada *push*.

¿Y ahora qué?

En el cuaderno **12_INTRO_GITHUB.ipynb**:

1.  **Tu Repositorio Remoto:** Crearás tu repositorio personal en la web de GitHub (<usuario>_PYTHON_CIENCIA_ABIERTA_2).
2.  **Vincular Local y Nube:** Conectaremos tu paquete local de Python con GitHub y subiremos tu primer commit oficial (git push).
3.  **Flujo de Ramas (*Feature Branch*):** Crearemos una rama de desarrollo (feature/edad-persona) para implementar nuevas reglas sin romper la rama principal.
4.  **Pull Request (PR):** Propondrás la integración de tus cambios a la rama principal simulando una revisión colaborativa.
5.  **Automatización con GitHub Actions:** Activaremos la Integración Continua (CI/CD) para que GitHub pase los tests de pytest automáticamente en la nube tras cada *push*.

¿Y ahora qué?



```
4. Paso 2: Sincronización del Paquete Local con el Repositorio Remoto

Ahora abrimos nuestra terminal integrada y vinculamos nuestra carpeta local con la dirección remota personalizada de GitHub.

Comandos de Configuración e Inicialización:

# 1. Nos aseguramos de estar en la carpeta de nuestro paquete
cd NOTEBOOKS/PYTHON_CIENCIA_ABIERTA_2

# 2. Configurar nuestra identidad global en Git (si no lo habíamos hecho antes)
git config --global user.name "Tu Nombre o Usuario"
git config --global user.email "tu_email@csic.es"

# 3. Inicializar el repositorio Git local (si no estuviera inicializado)
git init

# 4. Asegurarnos de que la rama principal se llama 'main'
git branch -M main

# 5. Vincular nuestro repositorio local con la URL de tu repositorio remoto en GitHub:
# Sustituye <user_name> por tu nombre de usuario real de GitHub
git remote add origin https://github.com/<user_name>/<user_name>_PYTHON_CIENCIA_ABIERTA_2.git

# Ejemplo real para el usuario 'guslicem':
# git remote add origin https://github.com/guslicem/guslicem_PYTHON_CIENCIA_ABIERTA_2.git

# 6. Añadir todos los archivos al área de preparación (staging)
git add .
```